

Variable Relaxation Parameter in Iterative Methods for Solving Sparse Linear Systems

Sean Meehan

December 11, 2025

Abstract

Classical iterative methods such as Jacobi and Gauss-Seidel are widely used for solving large, sparse linear systems due to their simplicity, but they often suffer from slow convergence. Introducing a relaxation parameter leads to the Weighted Jacobi and Successive Over-Relaxation (SOR) methods, which can significantly accelerate convergence when the parameter is well chosen. In this work, we investigate two further optimizations that employ variable relaxation parameters during the iteration process: the Scheduled Relaxation Jacobi (SRJ) method and the Adaptive SOR (A-SOR) method. We present the theoretical formulations of these schemes, describe their implementation for a sparse linear system arising from the finite-difference discretization of the 2D Laplacian, and compare their numerical performance against classical and weighted methods. Our results show that SRJ provides substantial reductions in iteration count, while the performance of A-SOR is highly sensitive to the update strategy and can be improved through appropriate bounds on the relaxation parameter. Overall, we demonstrate both the potential advantages and the practical limitations of variable-relaxation iterative schemes for solving large sparse systems.

1 Introduction

Iterative methods for solving linear systems are ubiquitous in scientific computing. Two classic iterative methods are the Jacobi and Gauss-Seidel methods. These methods are famous for their simplicity and ease of implementation. As such, they are typically given as the first example of iterative methods in numerical analysis texts. However, compared to modern methods like GMRES and multigrid, these classic methods have slow rates of convergence. In an effort to improve these convergence rates, we can introduce a relaxation parameter to the original methods. The resultant methods are known as Weighted Jacobi and Successive Over Relaxation (SOR), respectively. These methods can achieve significantly improved convergence rates with the proper choice of this relaxation parameter, but they still do not achieve convergence rates competitive with modern methods. In this paper, we will explore a further improvement to these classical methods. Rather than introducing a static relaxation parameter, we will explore the use of a variable relaxation parameter that changes over the course of the iterations. Particularly, we will consider the Scheduled Relaxation Jacobi (SRJ) and Adaptive SOR methods. The paper will consist of the following sections:

1. Introduction: this section.
2. Background: We will give a theoretical overview of Jacobi, Gauss-Seidel, Weighted Jacobi, and SOR.
3. Scheduled Relaxation Jacobi: Here we will introduce the SRJ Method.
4. Adaptive SOR: Here we will introduce the Adaptive SOR Method.
5. Implementation: Here we will discuss the implementation of both methods to a sparse linear system arising from a 2d discretized laplacian.
6. Numerical Results: Here we will present numerical results comparing the performance of both methods, along with several others mentioned in this paper, on the selected application.

7. Conclusion: We will summarize our most significant conclusions and discuss the benefits and limitations of both methods.

2 Background

We consider the linear system

$$A\vec{x} = \vec{b},$$

where $A \in \mathbb{R}^{m \times n}$ is sparse, $\vec{x} \in \mathbb{R}^n$ is unknown, and $\vec{b} \in \mathbb{R}^m$ is known. We can express A as:

$$A = L + D + U,$$

where L is strictly lower triangular, D is diagonal, and U is strictly upper triangular.

2.1 Jacobi

We can derive the Jacobi method as follows.

$$\begin{aligned} A\vec{x} &= \vec{b} \\ (L + D + U)\vec{x} &= \vec{b} \\ D\vec{x} &= -(L + U)\vec{x} + \vec{b} \\ \vec{x} &= -D^{-1}(L + U)\vec{x} + D^{-1}\vec{b} \end{aligned}$$

Now we define the iteration,

$$\vec{x}^{(k+1)} = -D^{-1}(L + U)\vec{x}^{(k)} + D^{-1}\vec{b}, \quad (1)$$

where $\vec{x}^{(k)}$ is the k^{th} approximation to the solution $\vec{x}$. [4]

2.2 Gauss-Seidel

The Gauss-Seidel method is derived similarly.

$$\begin{aligned} A\vec{x} &= \vec{b} \\ (L + D + U)\vec{x} &= \vec{b} \\ (L + D)\vec{x} &= -U\vec{x} + \vec{b} \\ \vec{x} &= -(L + D)^{-1}U\vec{x} + (L + D)^{-1}\vec{b} \end{aligned}$$

Now define the iteration,

$$\vec{x}^{(k+1)} = -(L + D)^{-1}U\vec{x}^{(k)} + (L + D)^{-1}\vec{b}, \quad (2)$$

where $\vec{x}^{(k)}$ is the k^{th} approximation to the solution $\vec{x}$. [4]

We can see that these methods are indeed simple and easy to implement. We can improve these methods by introducing a relaxation parameter ω . Consider the general iteration scheme,

$$\tilde{x}^{(k+1)} = G\vec{x}^{(k)} + \vec{c}, \quad (3)$$

where $G \in \mathbb{R}^{n \times n}$ and $\vec{c} \in \mathbb{R}^n$. We can see how (1) and (2) follow this form. We now define a new iteration scheme that allows us to introduce ω .

$$\vec{x}^{(k+1)} = \vec{x}^{(k)} + \omega(\tilde{x}^{(k+1)} - \vec{x}^{(k)}). \quad (4)$$

2.3 Weighted Jacobi

Viewing (3) and (4) in the context of (1), we can see that $\tilde{x}^{(k+1)} = -D^{-1}(L+U)\tilde{x}^{(k)} + D^{-1}\vec{b}$. Then (4) becomes,

$$\begin{aligned}\tilde{x}^{(k+1)} &= \tilde{x}^{(k)} + \omega(-D^{-1}(L+U)\tilde{x}^{(k)} + D^{-1}\vec{b} - \tilde{x}^{(k)}) \\ &= (1-\omega)\tilde{x}^{(k)} + \omega(-D^{-1}(L+U)\tilde{x}^{(k)} + D^{-1}\vec{b}) \\ &= ((1-\omega)I - \omega D^{-1}(L+U))\tilde{x}^{(k)} + \omega D^{-1}\vec{b}\end{aligned}$$

So our weighted Jacobi iteration is,

$$\tilde{x}^{(k+1)} = ((1-\omega)I - \omega D^{-1}(L+U))\tilde{x}^{(k)} + \omega D^{-1}\vec{b}. \quad (5)$$

2.4 Successive Over Relaxation

Viewing (3) and (4) in the context of (2), we can see that $\tilde{x}^{(k+1)} = -(L+D)^{-1}U\tilde{x}^{(k)} + (L+D)^{-1}\vec{b}$. Then (4) becomes,

$$\begin{aligned}\tilde{x}^{(k+1)} &= \tilde{x}^{(k)} + \omega(-(L+D)^{-1}U\tilde{x}^{(k)} + (L+D)^{-1}\vec{b} - \tilde{x}^{(k)}) \\ &= (1-\omega)\tilde{x}^{(k)} + \omega(-(L+D)^{-1}U\tilde{x}^{(k)} + (L+D)^{-1}\vec{b}) \\ &= ((1-\omega)I - \omega(L+D)^{-1}U)\tilde{x}^{(k)} + \omega(L+D)^{-1}\vec{b}\end{aligned}$$

So our SOR iteration is,

$$\tilde{x}^{(k+1)} = ((1-\omega)I - \omega(L+D)^{-1}U)\tilde{x}^{(k)} + \omega(L+D)^{-1}\vec{b}. \quad (6)$$

[4]

3 Scheduled Relaxation Jacobi

We can define a single step of SRJ as,

$$\tilde{x}^{(k+1)} = J_{k+1}(\omega_i, \tilde{x}^{(k)}) = ((1-\omega_i)I - \omega_i D^{-1}(L+U))\tilde{x}^{(k)} + \omega_i D^{-1}\vec{b}, \quad (7)$$

where ω_i is the i th relaxation parameter, given by

$$\vec{\omega} = \begin{bmatrix} \omega_1 \\ \omega_2 \\ \vdots \\ \omega_P \end{bmatrix} \quad \vec{q} = \begin{bmatrix} q_1 \\ q_2 \\ \vdots \\ q_P \end{bmatrix}$$

where q_i is the number of times ω_i is used in one cycle of M iterations. Note that $\sum_{i=1}^P q_i = M$. Then one cycle of SRJ can be expressed as the following sequence of successive operations:

$$J_1(\omega_1, \tilde{x}^{(0)}), \dots, J_{q_1}(\omega_1, \tilde{x}^{(q_1-1)}), J_{q_1+1}(\omega_2, \tilde{x}^{(q_1)}), \dots, J_{q_1+q_2}(\omega_2, \tilde{x}^{(q_1+q_2-1)}), J_{M-q_P+1}(\omega_P, \tilde{x}^{(M-q_P)}), \dots, J_M(\omega_P, \tilde{x}^{(M-1)}).$$

In words, we apply q_1 iterations using the relaxation parameter ω_1 , then q_2 iterations using ω_2 , and so on, until we have applied q_P iterations using ω_P . This completes one cycle of SRJ. We repeat as many cycles as is necessary to reach a desired tolerance.

We can see that any specific SRJ scheme is completely determined by the choice of the parameters P , $\vec{\omega}$, and $\vec{q}$. It is customary to refer to schemes as P -level SRJ schemes. For example, a 3-level SRJ scheme has $P = 3$ and uses 3 distinct relaxation parameters. As one might expect, the performance of any given SRJ scheme is heavily dependent on the choice of the parameters. In theory, these parameters could be chosen any way. However, some choices are better than others. The optimal choice for the parameters is nontrivial and dependent on the particular application that SRJ is being used for. SRJ was introduced in [6], and several sets of optimal parameters are given for variations of the 2D Laplace Equation with homogeneous Dirichlet boundary conditions. This work is expanded upon in [5] using higher level SRJ schemes.

4 Adaptive SOR

We can define the k th step of Adaptive SOR as,

$$\vec{x}^{(k+1)} = S(\vec{x}^{(k)}) = ((1 - \omega_k)I - \omega_k(L + D)^{-1}U)\vec{x}^{(k)} + \omega_k(L + D)^{-1}\vec{b}, \quad (8)$$

where ω_k is the value of the relaxation parameter during the k th step of the iteration. Notice here that we update the relaxation parameter at every step of the iteration. Adaptive SOR schemes vary widely in the way that new ω_k values are determined, but they usually depend on the residual value at the k th step. Let $\vec{r}^{(k)} = \vec{b} - A\vec{x}^{(k)}$ be that residual. We will give an overview of two common methods for updating ω here.

4.1 Line-Search Methods

These methods arise from viewing ω as a step size in an optimization scheme. The paper [2] gives an in depth treatment to this idea, and we will only summarize the results here:

$$\begin{aligned} h^{(k)} &= \frac{(\vec{r}^{(k)})^T \vec{r}^{(k)}}{(\vec{r}^{(k)})^T A \vec{r}^{(k)}} \\ \omega_k &= \frac{2h^{(k)}}{2 + h^{(k)}} \end{aligned} \quad (9)$$

This is a convenient, easy to implement solution to updating ω . The authors of [2] expand on their work and offer more complex, more efficient solutions relying on the same underlying ideas in [3].

4.2 Minimal Residual Methods

Minimal residual methods involve selecting the value of ω that will minimize $\vec{r}^{(k+1)}$ at each step. This is given in depth treatment in [1], and we will only summarize the results here. We know that

$$\vec{r}^{(k+1)} = \vec{b} - A\vec{x}^{(k+1)}.$$

But $\vec{x}^{(k+1)}$ is given by (8). Then we can express,

$$\vec{r}^{(k+1)} = \vec{b} - A(((1 - \omega_k)I - \omega_k(L + D)^{-1}U)\vec{x}^{(k)} + \omega_k(L + D)^{-1}\vec{b}).$$

So, we need to find the value of ω_k that minimizes this quantity. That is,

$$\omega_k = \operatorname{argmin}_{\omega} \|\vec{b} - A(((1 - \omega_k)I - \omega_k(L + D)^{-1}U)\vec{x}^{(k)} + \omega_k(L + D)^{-1}\vec{b})\|_2 \quad (10)$$

5 Implementation

We implemented all of the key methods mentioned in this paper in MATLAB. Pseudocode is not provided here, as all of the code associated with this paper will be attached at the end. The methods implemented are:

- Jacobi: implemented following (1).
- Gauss-Seidel: implemented following (2).
- Weighted Jacobi: implemented following (5). Optimal choice of ω found through experimentation.
- SOR: implemented following (6). Optimal choice of ω found through experimentation.
- Scheduled Relaxation Jacobi: implemented following (7). $\vec{\omega}$ and $\vec{q}$ taken from [5].
- Adaptive SOR using Line Search: implemented following (8) and (9).
- Adaptive SOR using Minimal Residual: implemented using (8) and (10).

All the methods mentioned in this paper are iterative solvers for sparse linear systems. The world is saturated with applications requiring such solvers. For this paper, to test these methods, we have chosen to use the discretized Laplacian in two dimensions with homogeneous Dirichlet boundary conditions. We have:

$$\begin{aligned} u_{xx} + u_{yy} &= f(x, y), \quad (x, y) \in [-1, 1] \times [-1, 1], \\ u(x = 1, y) &= 0, \\ u(x = -1, y) &= 0, \\ u(x, y = -1) &= 0, \\ u(x, y = 1) &= 0. \end{aligned}$$

It is well established that the discretization of this problem yields the linear system

$$A\vec{x} = \vec{b},$$

where $A \in \mathbb{R}^{n^2 \times n^2}$, $\vec{b}, \vec{x} \in \mathbb{R}^{n^2}$, and

$$A = \begin{bmatrix} T & -I & 0 & \cdots & 0 \\ -I & T & -I & \cdots & 0 \\ 0 & -I & T & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & -I \\ 0 & \cdots & 0 & -I & T \end{bmatrix}, \quad T = \begin{bmatrix} 4 & -1 & 0 & \cdots & 0 \\ -1 & 4 & -1 & \cdots & 0 \\ 0 & -1 & 4 & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & -1 \\ 0 & \cdots & 0 & -1 & 4 \end{bmatrix}.$$

By randomly generating $\vec{x}$, we created a sparse system to test the above methods.

6 Numerical Results

The first and most important question we must ask in our analysis of these iterative methods is how fast they converge. Figure 1 shows some expected results and some unexpected. We see that SRJ has

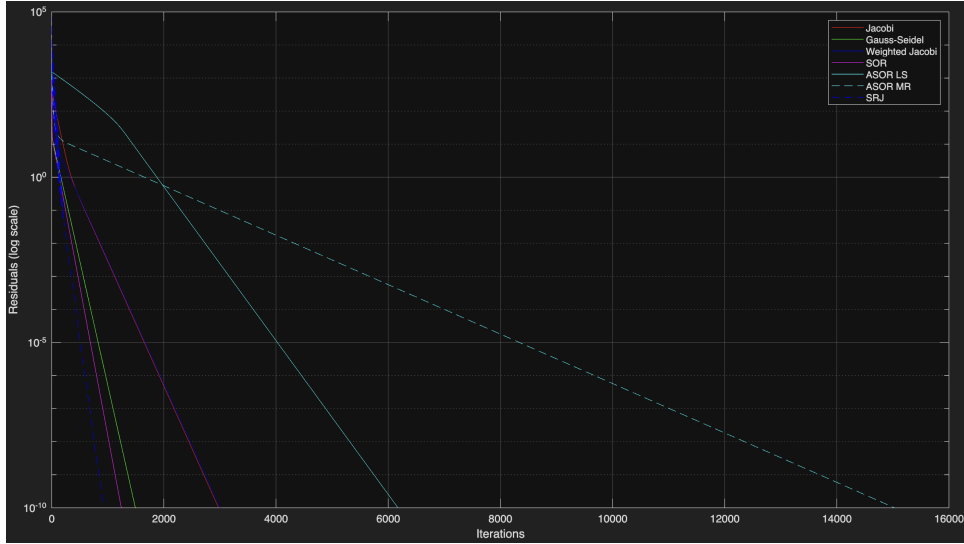


Figure 1: Residuals vs iteration for iterative methods

the fastest rate of convergence, which is unsurprising, as it is meant as an improvement of an improvement of Jacobi. Jacobi and Weighted Jacobi performed almost identically. SOR and Gauss-Seidel also performed similarly which is to be expected. Surprisingly both Adaptive SOR methods performed very poorly. They were outperformed by the methods they were invented to improve. We will demonstrate later in this section that both A-SOR methods can be improved with slight modifications. It is also true

that the performance of A-SOR is problem dependent. This may simply be the 'wrong problem' to use these methods on.

The next metric we are interested in is computational speed. Keeping Figure 1 in mind, the results

Method	Time (s)	Iterations
Jacobi	0.0580	2966
Gauss-Seidel	0.0284	1487
Weighted Jacobi	0.0559	2996
SOR	0.0230	1237
SRJ (2-level)	0.1875	929
Adaptive SOR (Line Search)	0.9946	6209
Adaptive SOR (Minimal Residual)	0.2811	14983

Table 1: Computational performance comparison of iterative methods

from Table 1 are to be expected. SRJ required the least iterations to converge, but it requires significantly more flops to complete one iteration of SRJ than it does to compute one iteration of Jacobi. So, despite requiring less iterations, it required more time than the simpler methods. The Adaptive SOR methods, which required significantly more iterations to converge than the other methods, took the longest (in time) as well, because they also require more flops than the simpler methods.

As we mentioned before, it is possible to improve the performance of the Adaptive SOR methods by making a slight modification. That modification is to bound the value of ω within a given range. That is, setting a range, and if ω is calculated to be outside that range, setting ω to the endpoint of the range. Numerical experimentation could yield the optimal range for ω for these methods, but for this paper, trial and error led to the use of the range $0.5 < \omega < 2$. Figure 2 Shows that this minor alteration produces

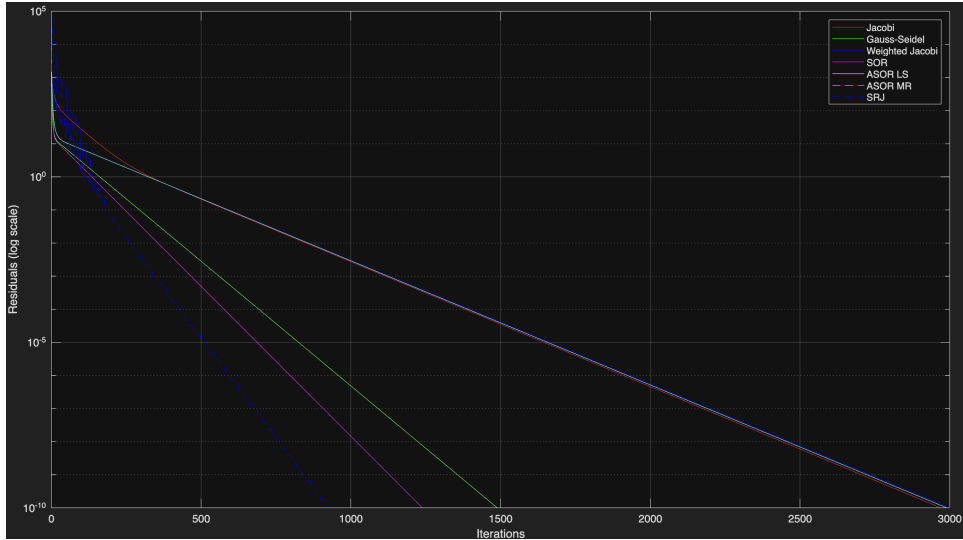


Figure 2: Residuals vs iteration for iterative methods (modified Adaptive SOR)

much better rates of convergence. Here both Adaptive SOR Schemes perform similarly to Jacobi and Weighted Jacobi. This is still rather unimpressive, but it is proof of concept that the schemes can be improved.

7 Conclusion

We have given an overview of two basic iterative methods, as well as several improvements on these iterations. We then implemented and tested these methods on a 2d discretized Laplacian system. The Adaptive SOR methods yielded surprisingly poor results, but did demonstrate potential for improvement. SRJ required the least iterations to converge, but took more time than some others. However, considering the paralelizability inherent to Jacobi methods, this computing cost could be distributed over several processors. This would almost certainly lead to SRJ having the lowest time to convergence. Then our conclusion is that, in a situation where parallel computing is possible, SRJ is the best choice for this particular problem. Where parallel computing is impossible, it is best to stick with a less computationally expensive method like SOR.

Bibliography

- [1] Y. Yuan, S. Sun, P. Chen, and M. Yuan, “Adaptive relaxation strategy on basic iterative methods for solving linear systems with single and multiple right-hand sides,” *Advances in Applied Mathematics and Mechanics*, vol. 13, no. 2, pp. 378–403, Jun. 2021. doi:10.4208/aamm.oa-2019-0378
- [2] Y. Miyatake, T. Sogabe, and S.-L. Zhang, “On the equivalence between sor-type methods for linear systems and the discrete gradient methods for Gradient Systems,” *Journal of Computational and Applied Mathematics*, vol. 342, pp. 58–69, Nov. 2018. doi:10.1016/j.cam.2018.04.013
- [3] Y. Miyatake, T. Sogabe, and S.-L. Zhang, “Adaptive Sor methods based on the Wolfe conditions,” *Numerical Algorithms*, vol. 84, no. 1, pp. 117–132, Jun. 2019. doi:10.1007/s11075-019-00748-0
- [4] R. Levesque, *Finite difference methods for ordinary and partial differential equations — siam publications library*, <https://epubs.siam.org/doi/book/10.1137/1.9780898717839> (accessed Dec. 11, 2025).
- [5] J. E. Adsuara, I. Cordero-Carrión, P. Cerdá-Durán, and M. A. Aloy, “Scheduled relaxation Jacobi method: Improvements and applications,” *Journal of Computational Physics*, vol. 321, pp. 369–413, Sep. 2016. doi:10.1016/j.jcp.2016.05.053
- [6] X. I. A. Yang and R. Mittal, “Acceleration of the Jacobi iterative method by factors exceeding 100 using scheduled relaxation,” *Journal of Computational Physics*, vol. 274, pp. 695–708, Oct. 2014. doi:10.1016/j.jcp.2014.06.010